

Foundations of R Chapter 2

Robert Gulotty

9/1/2018

Functions in R

In this chapter, we will learn how to make our own functions, building on the basic tools in R. In so doing, we will preview material in the second half of the course which has to do with mathematical models of probability and statistical inference. Just as the previous chapter, the goals here are not to offer a complete coverage, but rather to begin the process of familiarization with the terms of functions, probability, and statistical inference.

A simulated experiment.

R already contains a variety of random number generators, we will use one to simulate the small errors made over the course of an experiment or project.

Suppose we make mistakes in the course of a project or an exam. For example, if we are studying travel times, small events, a person unable to lift their bag into the overhead bin or person who was particularly efficient in finding their seat, can each contribute to a delay. We can model this process as each mistake causing a deviation up (+1) or down (-1) with equal probability, and these errors cumulate. Call this the small error process.

If we make n errors, than we can drift at most n away from the truth. If we have 10 errors, the chance of ten +1s in a row is:

```
1/2*1/2*1/2*1/2*1/2*1/2*1/2*1/2*1/2*1/2
```

```
## [1] 0.0009765625
```

Which is not very likely. How should we understand the overall chances that we end up with some aggregate error some distance away from 0 at the end of all these experiments?

In R we can analyze that process by constructing a function. First, we will make random draw of either (+1) or (-1). The set of all the possibilities from an experiment are called the **sample space**. Here we construct the sample space from a vector of 1 and -1.

```
samplespace <- c(1, -1)
```

If you just want a equal probability random sample without replacement, you can use the ‘sample’ function:

```
sample(samplespace, size = 1)
```

```
## [1] 1
```

The `sample()` function can generate all kinds of random events. The function is a computational example of a **random variable**. Mathematically, a random variable is a function that assigns a number to outcomes of some **sample space**. Here we could imagine that the 1 and -1 are assigned to the head and tail of a coin. Other examples of random variables might count the number of crashes in a car race, measure the time between bus arrivals at a bus stop, or the distance traveled by a cat in an evening. In each case there is some set of possible outcomes, and the random variable assigns a number to each outcome.

Our goal is to not determine the behavior of a single error, but to model the outcome of many cumulative errors. To do so, we will create a computation model by writing a function in R. Making your own function is as simple as naming an input, here `x`, writing some steps, and then specifying an output. The content of functions are embraced by `{` and `}`.

```
smallerror <- function(x) {  
  samplespace <- c(1, -1)  
  
  x <- x + sample(samplespace, size = 1)  
  # Once you are done, you can specify the value of the function as follows:  
  return(x)  
}  
  
smallerror(10)
```

```
## [1] 9
```

Because the `smallerror` function is a function of a random function, and returns a number, it is itself a random function. Later on we will learn that mathematical random functions can construct other random variables.

When should you write your own function? A rule of thumb is to write your own function when you find yourself copying and pasting the same code with only small changes.

Whether custom functions or not, all functions in R are written just like the above code. You can see what is inside such code by just typing the function in the console without ().

```
sample

## function (x, size, replace = FALSE, prob = NULL)
## {
##   if (length(x) == 1L && is.numeric(x) && is.finite(x) && x >=
##       1) {
##     if (missing(size))
##       size <- x
##     sample.int(x, size, replace, prob)
##   }
##   else {
##     if (missing(size))
##       size <- length(x)
##     x[sample.int(length(x), size, replace, prob)]
##   }
## }
## <bytecode: 0x7fa545939990>
## <environment: namespace:base>
```

Notice, there are four arguments to sample, and two of them have default settings. Inside the function is an if command, which allows the things in the curly braces to happen if the things in the parenthesis return TRUE, and the things that happen after the else if FALSE. These are called control functions.

Example of function:

```
euclideanDist<- function(x1, y1, x2, y2){
  dist=sqrt((x1-x2)^2+(y1-y2)^2)
  return(dist)
}
```

Example of control functions:

```
KnockKnock<- function(input){
  if(input=="Who's there?"){
    response=sample(c("Banana.", "Orange."),size = 1, replace = F, prob = c(.9,.1))
  }else if(input=="Banana who?"){
    response="Knock Knock."
  }else if(input=="Orange who?"){
    response="ORANGE YOU GLAD I DIDN'T SAY BANANA!"
  }else{
    response="PLAY ALONG!"
  }
  return(response)
}
```

For loops

We can apply our function repeatedly, and save the results in the i th element of a list called `errorpath`. This will tell us how far off we get across the experiment. To do so, we will use a special function called a `for` loop. A `for` loop iterates over the numbers listed, 1, 2, 3, ... 1000. Each loop increments i by 1. Notice the similarity to a function. The things in the curly brace are done repeatedly, from top to bottom.

```
for( k in 1:10){  
  print(k)  
}
```

```
## [1] 1  
## [1] 2  
## [1] 3  
## [1] 4  
## [1] 5  
## [1] 6  
## [1] 7  
## [1] 8  
## [1] 9  
## [1] 10
```

Note that unlike functions, the objects created in a loop exist outside the loop. This means you can save the output of each iteration of the loop in an object. It is generally good practice to create a vector, either of numbers or a list, of the same length as the loop to save the output.

```
Zerovector <- vector("numeric", 10)  
Nulllist <- vector("list", 100)
```

We can then insert something in each element of the vector by using the square bracket notation.

```
Zerovector <- vector("numeric", 10)  
for( k in 1:10){  
  
  Zerovector[k]<- k+10  
  
}  
Zerovector
```

```
## [1] 11 12 13 14 15 16 17 18 19 20
```

Often we plan on looping through an existing vector, rather than just manipulating the integers. For that we would still need a loop index, and we can easily construct one for any vector with `seq_along()`.

Application: looping smallerror

Now that we have some exposure to loops, we will use a loop to repeat a simulated experiment.

```
# We will start at 0  
start <- 0  
  
# We will calculate 1000 errors  
nerrors <- 1000  
  
# And we will save these results in "errorpath", a vector of length 1000  
errorpath <- vector("numeric", nerrors)
```

```

# Now we can write a for loop, which iterates over the numbers listed,
# here 1, 2, 3, ... 1000. Each loop increments i by 1.

for (i in seq_along(errorpath)) {
  # We start by putting start in the list errorpath at index 'i'

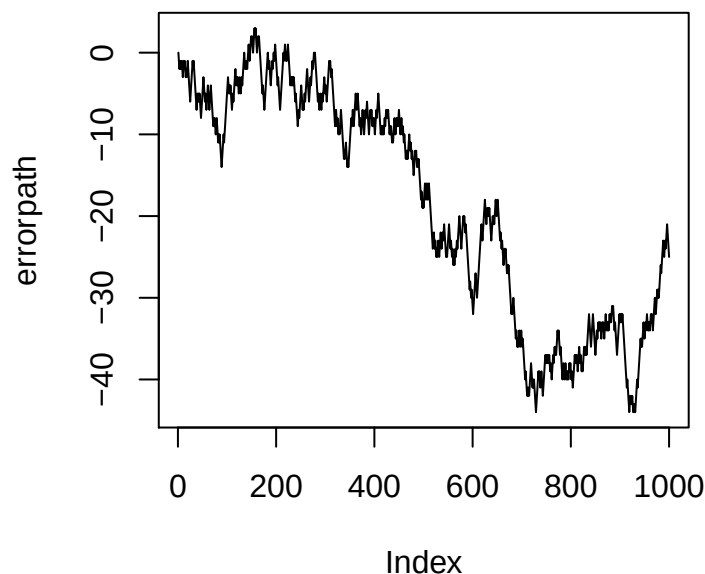
  errorpath[i] <- start

  start <- smallererror(start)
}

```

This created a recursive application of the smallererror function. To visualize the result, we will plot the data using a plot command with the option to draw a line.

```
plot(errorpath, type="l")
```



Overall, we see that the experiment drifts about. This is basically what happens to your total amount of winnings if you gamble in an even-odds game. If you stay long enough, you might get very rich, but you might get very poor, and once you hit zero you are stuck. That property makes gambling dangerous. In an experiment, our small errors lead us to drift about.

It turns out that this small error process has a name, it is called a random walk. It is very often used as a model for uncertainty. Its main use is as an assumption for where variation comes from in measurements, where we think that in addition to systematic effects of an experiment, lots of small cumulative errors affect the result. However, we only measure the cumulative effects of all these errors. If we made a bunch of small errors that we don't measure, and we want to know how likely is it that we get the right answer anyway, and how likely we are to get pushed away from the truth. In a sense, we want to know are we going to be like the gambler that loses it all. In this case, we drifted off by the amount listed at the last entry of 'errorpath'

```
length(errorpath)
```

```
## [1] 1000
```

```
errorpath[1000]
```

```
## [1] -25
```

What was the furthest we drifted?

```
max(abs(errorpath))
```

```
## [1] 44
```

That means that at some point we got 44 away from the 0. That is pretty far! This happened in iteration number 729, which you can find using the `which.max` function. It is good that we did not end the experiment there, or we would have really been off...

Simulating 10000 Experiments

To understand the properties of randomness, we can run simulations. Here we are going to run simulations to describe the properties of the cumulative effects of these little errors. Suppose we run ten thousand simulated experiments, each where there are 10,000 small errors, and each time record how far off we are from the truth. That will help us see how far we get from the real effects given these small errors. To do so, we will use a loop.

```
# Start like before, defining the parameters of our problem:

nexperiments <- 10000
nerrors <- 10000

# Note, we are going to make a new empty vector, and
# fill it with the end of each of our experiments.

outcomes <- vector("numeric", nexperiments)

for( j in 1: nexperiments){

  start<-0

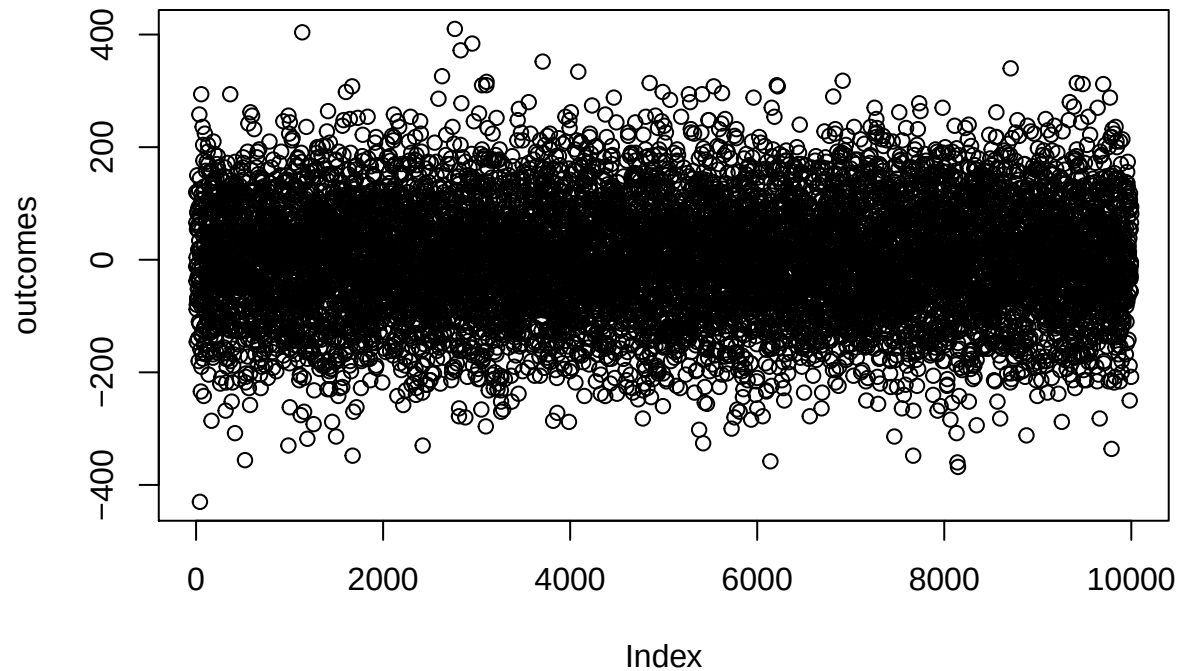
  # Now for each of the j experiments, we run 10000 small errors,
  # allowing the result to drift about.

  outcomes[ j ] <- sum(sample(c(-1,1), size = nerrors , replace=T))

}
```

Lets see what we got, plotting the points at the end of each of our experiments.

```
plot(outcomes)
```



They are centered close to zero, but how far does our experiment go wrong? We need to summarise the data. The quantile of a vector reports things like the median, the 95th percentile, ect.

```
quantile(outcomes, c(0.025, 0.05, 0.25, 0.5, 0.75, 0.95, 0.975))
```

```
##      2.5%      5%      25%      50%      75%      95%      97.5%  
## -196.05 -164.00  -68.00    0.00   68.00  164.00  192.00
```

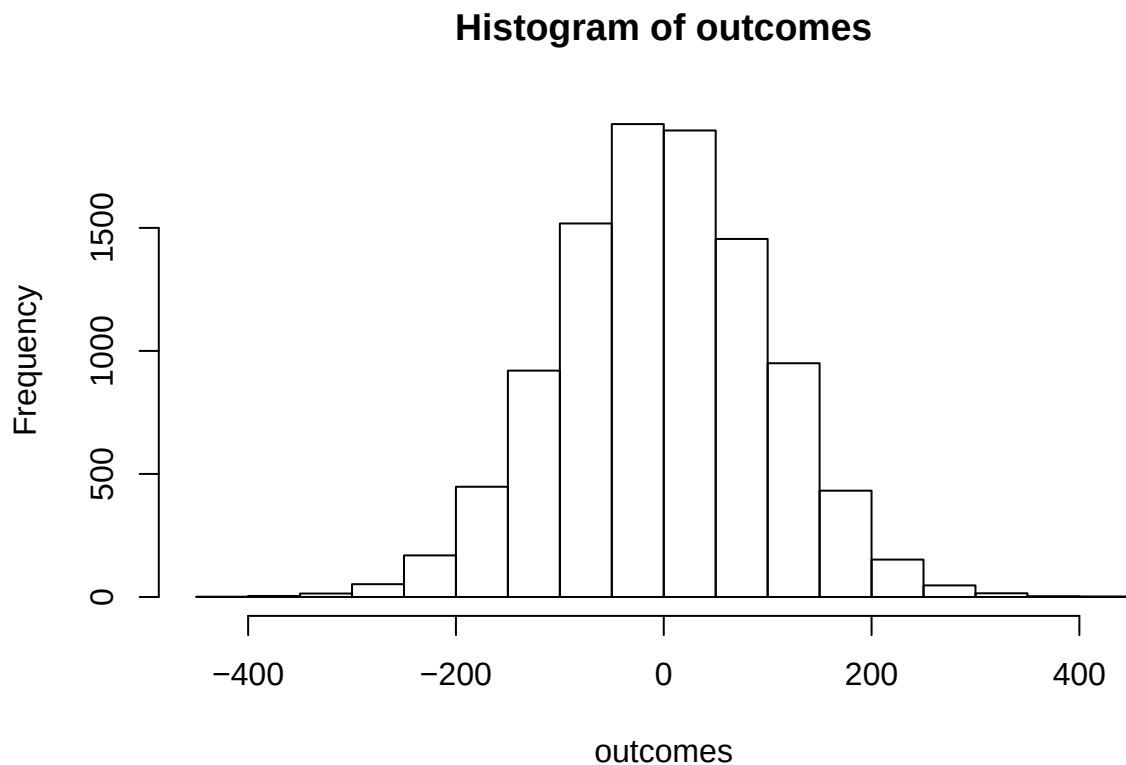
This means that 2.5% of the data is less than about -196, 2.5% of the data is above about 196. If we think about absolute value deviation, about 5% of the data is as far or further away than about 196.

```
mean(abs(outcomes)>=196)
```

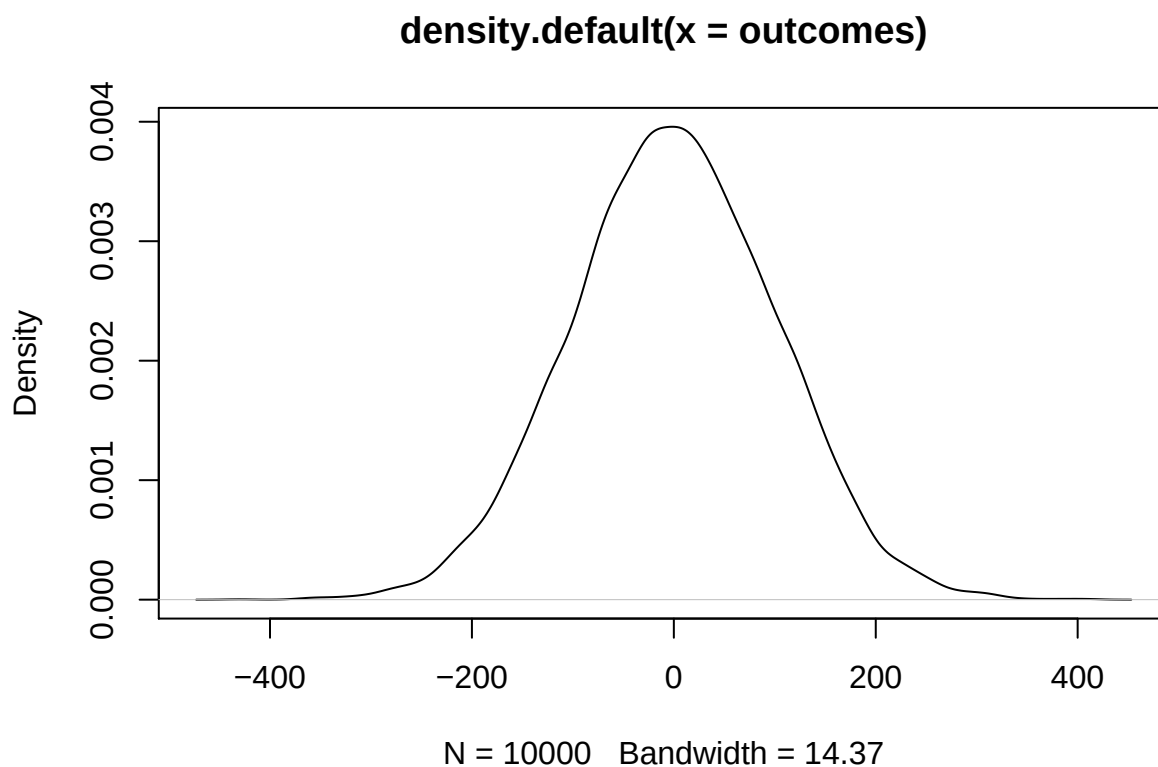
```
## [1] 0.0509
```


One way to visualize this data is to plot a histogram or density, which is a smoothed histogram.

```
hist(outcomes)
```

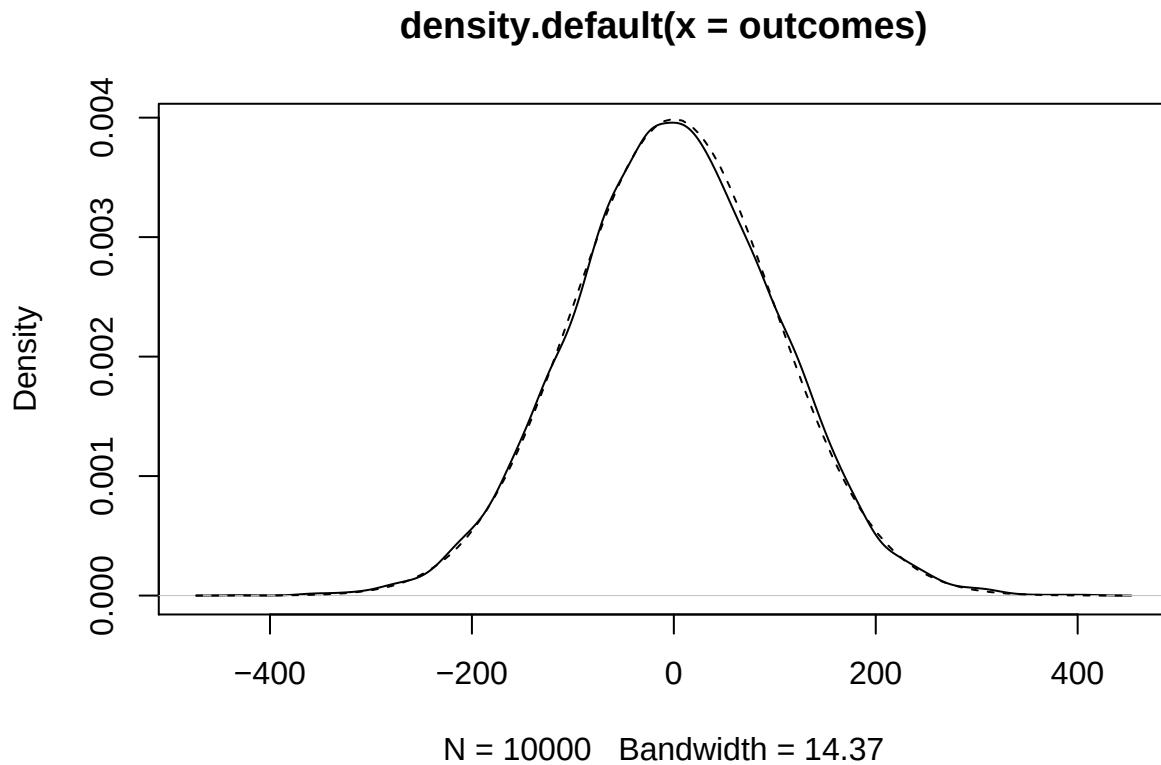


```
plot(density(outcomes))
```



Now it turns out that the function that best describes this curve is the following equation, that we will write into a function:

```
smallerrordensity <- function(x) {  
  sigma <- 100  
  (1 / sqrt(2 * pi * (sigma ^ 2))) * exp(-(x ^ 2) / (2 * (sigma ^ 2)))  
}  
  
plot(density(outcomes))  
  
curve(smallerrordensity, add = T, lty = 2)
```



R allows you to directly plot functions using the “curve” command. Run this after running the plot command above.

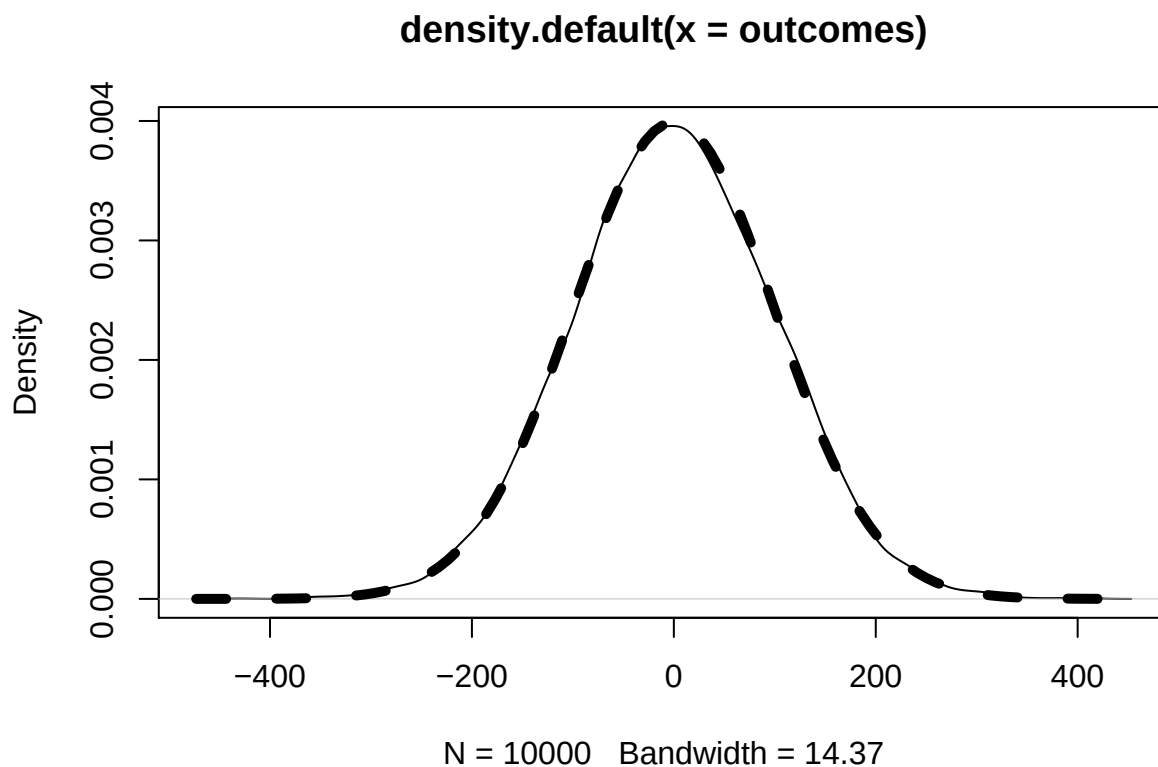
It turns out that this smallerror density has a name in statistics, the normal distribution, or gaussian distribution. The height of the function at any given point is given by the equation in the small error density:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$$

Because this is such a popular function, R includes it in its base package and calls it `dnorm`, which stands for the probability density of the normal distribution.

```
plot(density(outcomes))
```

```
curve(dnorm(x = x, sd = 100), add = T, lty = 2, lwd = 5)
```



Preview of probability density functions

In R, we have several ways to get at the “distribution” of random variables.

`dnorm` is the normal probability density function $f_X(x)$, such that $P(a \leq X \leq b) = \int_a^b f_X(x)dx$. In words, this means that the probability of getting some outcome X between the numbers a and b is the area under the curve of $f_X(x)$ between a and b .

We will learn about the properties of this function later, but let's use R to discover some of them now.

A function can be a probability density function if it has the following properties:

- The function is always non-negative.
- The area under the function adds up to 1.

The following code examines whether the `smallerrordensity` is positive, and it uses the computer to calculate the area under the curve, using the `integrate` function, which you will learn about later.

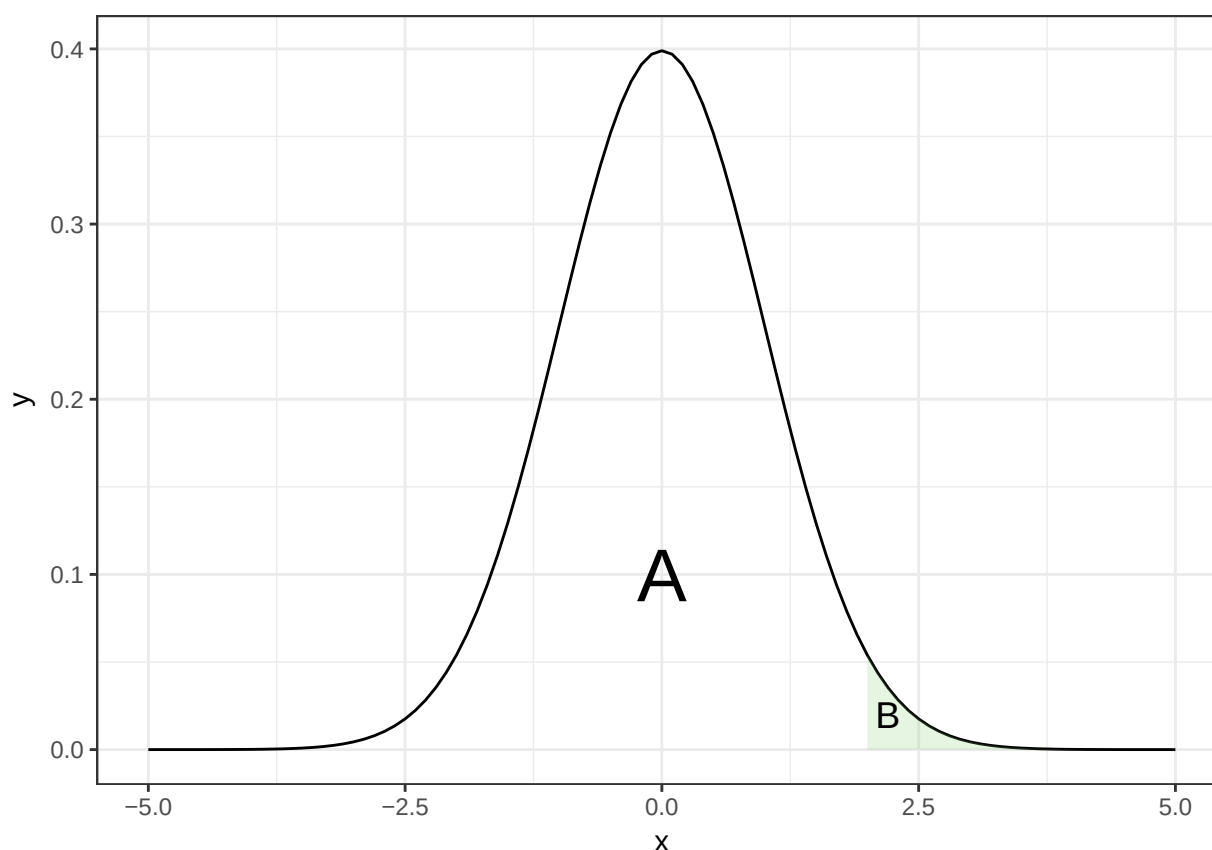
```
table(smallerrordensity(-10:10) >= 0)

##
## TRUE
##    21

integrate( smallerrordensity, lower= -Inf, upper=Inf)

## 1 with absolute error < 5.1e-05
```

Interpreting the area under the pdf



We can calculate how likely we are to end up in the tail of the distribution by using this integration function. Lets see how much area is covered by the function left of 196. In the figure above, this is the non-shaded region A.

```
integrate( smallerrordensity, lower= -Inf, upper = 196)
```

```
## 0.9750021 with absolute error < 6.5e-05
```

The area to the right of 196, the area B is just 1 minus this number.

```
integrate( smallerrordensity, lower= 196, upper = Inf)
```

```
## 0.02499759 with absolute error < 4.5e-05
```

Because the function is symmetric, we could combine the areas in the tails

```
higherrorprob <- integrate( smallerrordensity, lower= 196, upper= Inf)
lowerrorprob <- integrate( smallerrordensity, lower= -Inf, upper= -196)
```

```
higherrorprob$value+lowerrorprob$value
```

```
## [1] 0.04999518
```

These integrals create a function called the **cumulative distribution function** (cdf), which is the area under the density curve to the left of some point point x.

`pnorm()` is the **Normal cumulative density function**.

Compare the previous calculations with the following:

```
pnorm(-1.96)*2
```

```
## [1] 0.04999579
```

To get the exact value of at which the area under the tails is equal to .05, we can use the qnorm function. Note that this is very close to 1.96

```
qnorm(.025)
```

```
## [1] -1.959964
```

```
qnorm(.975)
```

```
## [1] 1.959964
```

To summarise, we have modeled a small error process, cumulated those errors, and derived the normal distribution. The normal distribution can be described mathematically with its pdf $f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2}{2\sigma^2}}$, which, when integrated over with bounds a and b, gives us the probability that our experiments is off by a number between a and b.

Preview of Statistical Inference

Lets see if we can use these patterns to do statistical inference. We earlier suspected that departure delay was associated with small mistakes by passengers. One issue is that on small planes there is less room for these little errors: people often have trouble finding space for all their stuff. In the following, we test whether planes that are smaller experience more departure delays than planes that are bigger.

We begin by load the data from the previous R exercise.

```
library(dplyr, warn.conflicts = F)
library(nycflights13)
```

```
## Warning: package 'nycflights13' was built under R version 3.4.4
```

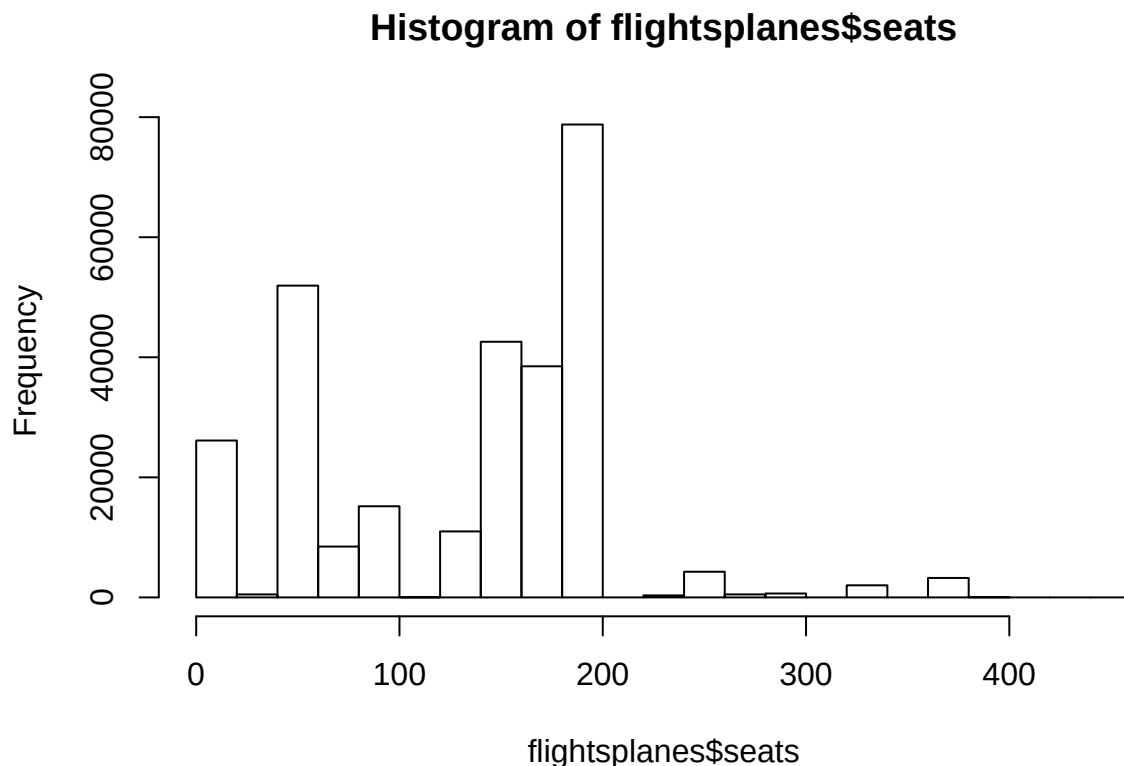
To know how many seats are on a plane, we join the flights data with the plane data. This is a step that we will describe in more detail in later chapters.

```
flights <- select(flights, -year)
flightsplanes<- left_join(flights, planes)
```

```
## Joining, by = "tailnum"
summary(flightsplanes$seats)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's
##       2.0   55.0   149.0   136.7   189.0   450.0   52606
```

```
hist(flightsplanes$seats)
```



Some flights have 2 seats, some have 450. It looks like there are four kinds of planes: 2 seaters, sub 100 seaters, between 100 and 200 and above 200 seat planes. For simplicity, we will select only planes between 100 and 210 seats, and see if those that are above 150 have more delay than those that are below 150. We then take a subsample (one tenth) to later do validation.

```

flightsplanesall<- left_join(flights, planes)

## Joining, by = "tailnum"
flightsplanesall<-filter(flightsplanesall, seats>100&seats<210)
flightsplanesall<-mutate(flightsplanesall, bigplane=ifelse(seats>150, "bigger", "smaller"))
set.seed(100)

trainsample <- sample(dim(flightsplanesall)[1], size =dim(flightsplanesall)[1]/10 )

flightsplanes<-flightsplanesall[trainsample,]

```

This is a great example of classic assign manipulate assign formulation. At each stage we rewrote an object in R (`flightsplanesall`) with a slightly manipulated version of the object. A useful trick that is used in later chapters is the `%>%` or “pipe” notation. The following section briefly introduces this pipe notation.

Intro to Pipes

The default behavior of `%>%` is to place the left hand object as the first argument, i.e. `x %>% f(y)` is equivalent to `f(x, y)`. Consider the analysis above in pipe notation:

```

flightsplanesall<-flights%>%
  left_join(planes)%>%
  filter(seats>100&seats<210)%>%
  mutate( bigplane=ifelse(seats>150, "bigger", "smaller"))

```

```
## Joining, by = "tailnum"
```

The notation is more compact, and can be read in words, take flights, join it with planes, filter by seat size, and then create a new variable called `bigplane`, assign the remainder to `flightsplanesall`.

Estimators

To summarise the difference in delay, we will write an *estimator*. An *estimator* is a function of the data, and produces a number or set of numbers, so it is a *random variable*. The output of an *estimator* is an *estimate*, and these are used to infer the value of an unknown parameter. Here our *estimator* is the familiar average.

```

average <- function(x){
  x<- x[!is.na(x)]
  averageout<-sum(x)/length(x)
  return(averageout)
}

```

```

flightmeans <- group_by(flightsplanes, bigplane)
(flightmeans <- summarise(flightmeans, meandep_delay = average(dep_delay)))

```

```

## # A tibble: 2 x 2
##   bigplane meandep_delay
##   <chr>         <dbl>
## 1 bigger         10.2
## 2 smaller        12.2

```


The difference between the two groups is :

```
(meandiff <- flightmeans$meandep_delay[2]-flightmeans$meandep_delay[1])
```

```
## [1] 2.085621
```

Or, put in words, big planes experience 2.1 minutes less delay.

Introduction to statistical models

But how do we know that 2.1 minutes is a systematic difference? How can we account for the statistical uncertainty inherent in flights? What we need is a statistical model. Our statistical model will assume that the overall population of flight times is subject to the kinds of small errors described above, and that under the *null hypothesis*, the two groups are drawn from such distributions with equal means. As analysts, we will want to be able to say more than the data are inconsistent with no difference, but let's start here.

In order to say what sorts of differences the model under the *null hypothesis* would generate, we need to make a claim about the spread of the distribution. The small error distribution, or standard normal distribution, assumes that the small errors are all of size 1. The properties of the random walk tell us that the probability of drift scales with $\sqrt{n}$. To use this model to describe flights, we need to put them on the same scale, which means correcting for the spread. Unfortunately, we only get to know the spread from the data, so we need another estimator.

Rather than coming up with two estimates of the spread, we will pool the data from both groups to calculate one spread. We often do not have a theory for why the spread would be different, so it is not unreasonable to operate under the assumption that they are the same. Nonetheless, we need to estimate this spread from the data, using a second **estimator**, here called the **standard_deviation**.

```
standard_deviation <- function(x){  
  
  x<- x[!is.na(x)]  
  
  sdout<-sqrt(sum((x-average(x))^2)/(length(x)-1))  
  return(sdout)  
}  
  
flightsd <- filter(flightsplanes,!is.na(bigplane))  
flightsd <- group_by(flightsd, bigplane)  
flightsdout <- summarise(flightsd ,  
                          sdflighttime = standard_deviation(dep_delay),  
                          ngroup = n())  
  
flightsdout  
  
## # A tibble: 2 x 3  
##   bigplane sdflighttime ngroup  
##   <chr>      <dbl>   <int>  
## 1 bigger      35.3   11760  
## 2 smaller     39.3    5328
```

The sample standard deviation or spread of the bigger planes is 35.28, the smaller planes standard deviation is 39.33

We can combine these two estimates into a single estimate of the spread by using the weighted sum, and then normalizing `meandiff` by this number. Because it is a weighted sum, the result is essentially the same as the standard deviation of the non-christmas flights, but below is the general formula you will use in the second half of the course:

$$S_{diff} = \sqrt{\left(\frac{(n_1 - 1)s_1^2 + (n_2 - 1)s_2^2}{(n_1 - 1) + (n_2 - 1)}\right) \left(\frac{1}{n_1} + \frac{1}{n_2}\right)}$$

Here we write our own function which produces this pooled standard deviation.

```
sd1 <- flightsdout$sdfighttime[1]
sd2 <- flightsdout$sdfighttime[2]
sn_1 <- flightsdout$ngroup[1]
sn_2 <- flightsdout$ngroup[2]

sample_SD <- function(sd1, sd2, sn_1, sn_2) {
  SD2 <- ((sn_1 - 1) * (sd1 ^ 2) + (sn_2 - 1) * (sd2 ^ 2)) / (sn_1 + sn_2 - 2)
  SD2norm <- sqrt(SD2) * (sqrt(1 / sn_1 + 1 / sn_2))
  return(SD2norm)
}

statistic <- meandiff / (sample_SD(sd1, sd2, sn_1, sn_2))
statistic
```

```
## [1] 3.451079
```

The $\sqrt{SD2}$ part of the pooled spread is 36.59, which between the standard deviation of the two classes of planes. The resulting normalized difference, `statistic`= 3.45 is a random variable that is distributed in a way that is very similar to a normal distribution. Technically, this statistic is a *t statistic*, and follows a **t-distribution**.

Definition and usage of p-values

This statistic is now standardized, and we can compare to our small error experiment. We do so by asking how often our simulation, which assumes that positive and negative errors are equally likely, but that the average is 0, none the less generates differences beyond those we observe in the data, summarized by our mean difference statistic.

```
mean(abs(outcomes)>= statistic*100)
```

```
## [1] 0.0012
```

The share of times that our small error model is more extreme is very small. 0.12 percent of the time, the difference we observe arose in our simulation **by chance**. Note that this number depended on our simulation as well as our data.

This number, the share of simulations that are larger than our observed difference, is the infamous “p-value”. A p-value is defined as the probability under a specific statistical model (here the small-error model of equal means) that a statistical summary of the data (here the difference in means) would be equal to or more extreme than the observed value.

Again, a p-value is the *probability*, under a model, that some estimator is *equal to or larger* than *the estimate from our data*.

P-values are great because they offer a continuous summary for how inconsistent a set of data is with a proposed model. The smaller the p-value, the greater the statistical incompatibility of the data with the proposed model. Here, the proposed model is that our data follows a small error distribution and there is no difference between the mean flight times. We find a small p-value, or evidence against that model having generated our data. Notice that either the distributional assumption, or the hypothesized equality of means, could be the cause of the incompatibility between the model and the data.

P-values are often misused: a large p-value does not indicate consistency of the model with the data, as there can be other models that are more consistent with the data. Just because a P-value is small does not mean that the effect is substantively large, again, that would be inferring that something is true, as opposed to not-yet-falsified.

Because the small error model with large sample sizes follows what is known as the normal distribution, R has built in functions for the exact answer.

```
(1-pnorm(statistic))*2
```

```
## [1] 0.0005583495
```

Another way to see this is that the statistic has been rescaled to the small error distribution where the ‘sigma’ or standard deviation is 1.

```
smallerrordensity <- function(x) {  
  sigma <- 1  
  (1 / sqrt(2 * pi * (sigma ^ 2))) * exp(-(x ^ 2) / (2 * (sigma ^ 2)))  
}  
  
area <- integrate(smallerrordensity, lower = statistic, upper = Inf)  
area$value * 2
```

```
## [1] 0.0005583496
```

It turns out that the area under the curve of the smallerror density in the tails past the statistic is this value.

So we can say, that if our model is right, and flights are governed by small errors, then we would expect see a difference in means between these two populations at least as large as 25 minutes about half a percent of the time. We thereby have evidence against flights out of New York on Christmas Eve being the same, on average, as other evenings in terms of average length.

Two sample t tests in base R

Everything we have done so far will be explored in more detail later in the course. Right now, you should be familiar with functions. In practice, we would in fact calculate whether two samples have a statistically distinguishable mean with the following command:

```
bigplanetrain <- filter(flightsplanes, bigplane=="bigger")
smallplanetrain <- filter(flightsplanes, bigplane=="smaller")

t.test(bigplanetrain$dep_delay, smallplanetrain$dep_delay, var.equal = T)

##
## Two Sample t-test
##
## data: bigplanetrain$dep_delay and smallplanetrain$dep_delay
## t = -3.4422, df = 17006, p-value = 0.0005784
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -3.2732400 -0.8980016
## sample estimates:
## mean of x mean of y
## 10.15371 12.23934
```

The `t.test` reports a t statistic, which was our statistic. The p-value is the p-value calculated with the t-distribution, which is very similar to the normal distribution derived above when the sample size is large. As we can see, the `t.test` reports the same numbers as our analysis above.

It is generally good practice to hold out a sample to validate an analysis, here we use the remaining data to test whether these differences persist.

```
flightsplanetest<-flightsplanesall[-trainsample,]
bigplanetest <- filter(flightsplanetest, bigplane=="bigger")
smallplanetest <- filter(flightsplanetest, bigplane=="smaller")

t.test(bigplanetest$dep_delay, smallplanetest$dep_delay, var.equal = T)

##
## Two Sample t-test
##
## data: bigplanetest$dep_delay and smallplanetest$dep_delay
## t = -8.7256, df = 152950, p-value < 2.2e-16
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
## -2.237428 -1.416640
## sample estimates:
## mean of x mean of y
## 10.68600 12.51303
```

Finally, all of the analysis we have done can be done with a linear model, which you will learn about in subsequent chapters.

```
modout<-lm(dep_delay~bigplane, data=flightsplanetest)
summary(modout)

##
## Call:
## lm(formula = dep_delay ~ bigplane, data = flightsplanetest)
##
```

```
## Residuals:
##      Min       1Q   Median       3Q      Max
## -53.69 -15.69 -12.51  -2.51  947.49
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    10.6860     0.1173  91.133  <2e-16 ***
## bigplanesmaller  1.8270     0.2094   8.726  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 37.99 on 152950 degrees of freedom
## (844 observations deleted due to missingness)
## Multiple R-squared:  0.0004975, Adjusted R-squared:  0.000491
## F-statistic: 76.14 on 1 and 152950 DF, p-value: < 2.2e-16
```

Recap

This chapter walked through an analysis, building a statistical model and then deploying it to study a basic empirical question. Your own work will likely skip some of these steps (such as re-deriving the normal distribution) but it will implicitly go through each step of the process. The key lessons are to identify a data-generating process, write down a model that captures this process, namely what process is stochastic and what is systematic, and then use the model to draw inferences that can distinguish the signal from the noise.

Problems

- (1) Write your own “quantile” function which gets the median and the 95th percentile of data.
- (2) What does `df` look like when it is printed out?

```
df <- data.frame(dogheight=c("10", "20", "30"), dogwidth=c("5", "3", "1"))  
df[df$dogwidth==3,1] <- NA
```

- (3) *True or False*
 - (a) A small p-value, such as .001 or .0001, means that our estimate is unlikely to have arisen from the null hypothesis.
 - (b) The small errors distribution is a nice approximation for the distances traveled by flights out of New York City.
 - (c) $f(x) = 2x$ for $0 \leq x \leq 1$ is a possible probability density function.
 - (d) The probability that a normal distribution produces the number 0 is 0.